JAVA PROGRAMMING

Abstract Classes, Interfaces, Packages & Access Modifiers

Easy English Notes • Beginner Friendly • University Exam Preparation

Topics covered

- Abstract classes and abstract methods
- Interfaces, implementation and inheritance
- Default, static and private interface methods
- Functional and marker interfaces
- Abstract class vs interface
- Packages, imports and static imports
- Access modifiers, classpath and naming conventions

Read the definitions first, then study the small Java programs and revision tables.

Prepared as clear, printable A4 study notes

1. Abstract Classes and Abstract Methods

Abstract class: A class declared with the `abstract` keyword is called an abstract class. It is used as a base class and cannot be directly instantiated (we cannot create its object).

Abstract method: A method declared without a body is called an abstract method. The child class must implement it unless the child class is also abstract.

```
abstract class Shape {
    abstract void draw();   // abstract method

    void message() {        // normal method
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing circle");
    }
}

class Main {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.message();
        s.draw();
    }
}
```

Output: Drawing a shape
Drawing circle

Important points

• An abstract class cannot be instantiated directly.

• An abstract class can contain abstract methods and normal methods.

• It can have constructors, variables and static methods.

• A subclass must implement all inherited abstract methods, or it must also be declared abstract.

• An abstract class can have zero abstract methods; it may still be declared abstract to prevent direct object creation.

Remember: Abstract classes are useful when related classes share common data or common code, but some behaviour must be supplied by child classes.

2. Interface Declaration and implements

An **interface** is a reference type that defines a contract. It tells a class which methods it should provide. A class uses the `implements` keyword to implement an interface.

```
interface Printable {
    void print();       // public abstract by default
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing report");
    }
}

class Main {
    public static void main(String[] args) {
        Report r = new Report();
        r.print();
    }
}
```

Output: Printing report

Basic interface syntax

```
interface InterfaceName {
    // constants and method declarations
}

class ClassName implements InterfaceName {
    public void methodName() {
        // method body
    }
}
```

- A class implements an interface using the `implements` keyword.
- A class must implement all required abstract methods, or be declared abstract.
- Interface abstract methods are implicitly public, so implementations must be public.
- Interface fields are implicitly public, static and final (constants).

Interface reference

```
Printable p = new Report();
p.print();
```

An interface reference can refer to an object of any class that implements that interface. This supports polymorphism.

3. Multiple Interface Implementation

Java does not allow a class to extend two classes, but one class can implement more than one interface. This is called multiple interface implementation.

```
interface Camera {
    void takePhoto();
}

interface MusicPlayer {
    void playMusic();
}

class Phone implements Camera, MusicPlayer {
    public void takePhoto() {
        System.out.println("Photo taken");
    }

    public void playMusic() {
        System.out.println("Music playing");
    }
}

class Main {
    public static void main(String[] args) {
        Phone p = new Phone();
        p.takePhoto();
        p.playMusic();
    }
}
```

Output: Photo taken
Music playing

Benefit: A class can follow multiple contracts without inheriting implementation from multiple classes.

4. Interface Inheritance

An interface can inherit another interface by using the `extends` keyword. An interface can extend more than one interface.

```
interface Printable {
    void print();
}

interface Scannable {
    void scan();
}

interface MultiFunction extends Printable, Scannable {
    void fax();
}

class Machine implements MultiFunction {
    public void print() {
        System.out.println("Print");
    }

    public void scan() {
        System.out.println("Scan");
    }

    public void fax() {
        System.out.println("Fax");
    }
}
```

Here, `MultiFunction` inherits the methods of both Printable and Scannable, and Machine implements all three methods.

5. Default, Static and Private Interface Methods

Modern Java interfaces can contain different kinds of methods. Default and static methods were introduced in Java 8. Private interface methods were introduced in Java 9.

A. Default method

A default method has a body and uses the `default` keyword. An implementing class inherits it and may override it.

```
interface Greeting {
    default void sayHello() {
        System.out.println("Hello!");
    }
}

class User implements Greeting {
}

class Main {
    public static void main(String[] args) {
        new User().sayHello();
    }
}
```

B. Static method

A static interface method belongs to the interface itself. Call it using the interface name.

```
interface MathTool {
    static int square(int n) {
        return n * n;
    }
}

class Main {
    public static void main(String[] args) {
        System.out.println(MathTool.square(5));
    }
}
```

Output: 25

C. Private method

A private interface method is a helper method used inside the interface to avoid repeating code. It is not directly accessible by implementing classes.

```
interface Greeting {
    private void message() {
        System.out.println("Welcome");
    }

    default void greet() {
        message();
    }
}

class User implements Greeting {
}
```

Method type	Main use	How it is called
Abstract	Declares required behaviour	Implemented by class
Default	Provides inherited method body	Through implementing object
Static	Utility method of interface	InterfaceName.method()

Method type	Main use	How it is called
Private	Shares helper code inside interface	Only inside that interface

6. Functional and Marker Interfaces

A. Functional interface

A functional interface is an interface that has exactly one abstract method. It can have any number of default, static or private methods. It is commonly used with lambda expressions.

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}

class Main {
    public static void main(String[] args) {
        Calculator c = (a, b) -> a + b;
        System.out.println(c.add(10, 20));
    }
}
```

Output: 30

The `@FunctionalInterface` annotation is optional, but it asks the compiler to check that the interface follows the functional-interface rule.

B. Marker interface

A marker interface is an interface with no methods or constants. It is used to mark a class so that the Java platform or a framework can recognise a special capability.

Examples: Serializable and Cloneable.

```
import java.io.Serializable;

class Student implements Serializable {
    int rollNo;
    String name;
}
```

Here, Student is marked as Serializable. The interface itself does not add a method that Student must implement.

7. Abstract Class vs Interface

Abstract class	Interface
Declared using abstract class	Declared using interface
A class extends one class only	A class can implement many interfaces
Can have instance variables	Fields are public static final constants
Can have constructors	Cannot have constructors
Can have abstract and concrete methods	Can have abstract, default, static and private methods
Can use different access modifiers	Abstract methods are public
Useful for shared state and common base code	Useful for defining a common contract

When to use an abstract class? Use it when closely related classes need shared fields, constructors or common implementation.

When to use an interface? Use it when different classes need to promise the same behaviour, even if they are not closely related.

8. Packages in Java

A package is a group of related classes, interfaces and sub-packages. It helps organise code and avoid name conflicts.

Types of packages

• **Built-in packages:** Supplied by Java, for example java.lang, java.util, java.io and java.time.

• **User-defined packages:** Created by programmers to organise their own classes.

Creating a package

```
package mypack;

public class Message {
    public void show() {
        System.out.println("Hello from package");
    }
}
```

Using a package

```
import mypack.Message;

class Main {
    public static void main(String[] args) {
        Message m = new Message();
        m.show();
    }
}
```

The package declaration, if present, must be the first non-comment statement in the source file. The folder structure normally matches the package name, for example mypack/Message.java.

9. import and static import

Normal import

The `import` statement lets you use a class or interface from another package without writing its full package name every time.

```
import java.util.Scanner;
import java.util.*;  // imports accessible types in java.util

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int age = sc.nextInt();
        System.out.println(age);
        sc.close();
    }
}
```

Importing java.util.* does not automatically import classes from its sub-packages.

Static import

Static import allows static members to be used without writing the class name each time.

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

class Main {
    public static void main(String[] args) {
        System.out.println(PI);
        System.out.println(sqrt(25));
    }
}
```

Use static imports carefully. Too many can make it difficult to identify where a member comes from.

10. Access Modifiers

Access modifiers control where a class, variable, constructor or method can be accessed from.

Modifier	Same class	Same package	Subclass outside package	Other package
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No*	No
protected	Yes	Yes	Yes*	No**
public	Yes	Yes	Yes	Yes

* Default access is not available to a subclass merely because it is a subclass in another package. ** Protected access outside the package is available to subclasses under Java's protected-access rules; it is not general access for every unrelated class.

A. private

Accessible only inside the same top-level class (including its nested members where permitted).

```
class Account {
    private int pin = 1234;

    void showPin() {
        System.out.println(pin);
    }
}
```

B. default (package-private)

When no access modifier is written, the member has default or package-private access. It can be accessed by classes in the same package.

```
class Student {
    int marks = 90; // default access
}
```

C. protected

Accessible within the same package and also by subclasses in other packages, subject to Java's protected access rules.

```
class Parent {
    protected void display() {
        System.out.println("Protected method");
    }
}
```

D. public

Accessible from any class that can access the declaring type.

```
public class Student {
    public void show() {
        System.out.println("Public method");
    }
}
```

Important: A top-level class can be public or package-private. It cannot be declared private or protected.

11. Classpath

Classpath is the path or list of locations where Java tools and the JVM look for user classes, packages and JAR files.

Classpath entries may be directories containing compiled .class files, JAR files, or other supported locations.

Using classpath

```
# Windows Command Prompt
javac -cp .;lib\mylibrary.jar Main.java
java -cp .;lib\mylibrary.jar Main

# macOS / Linux
javac -cp .:lib/mylibrary.jar Main.java
java -cp .:lib/mylibrary.jar Main
```

In Windows, classpath entries are separated by a semicolon (;). On macOS and Linux, they are separated by a colon (:). The dot (.) means the current directory.

Note: The `-cp` or `-classpath` option sets the classpath for that command. If no classpath is supplied, Java uses its default classpath behaviour.

12. Java Naming Conventions

Naming conventions are common rules for writing readable and consistent Java code. They are conventions, not strict language rules.

Item	Convention	Example
Class name	PascalCase	StudentRecord
Interface name	PascalCase	Runnable
Method name	camelCase	calculateTotal()
Variable name	camelCase	studentName
Constant	UPPER_SNAKE_CASE	MAX_SIZE
Package	lowercase	com.school.app
Enum type	PascalCase	Day
Type parameter	Usually one uppercase letter	T, E, K, V

Good names explain the purpose of the code. Avoid unclear names such as x1, abc or temp when a meaningful name is possible.

13. Quick Revision Sheet

Topic	Key point
Abstract class	Cannot be instantiated directly; may contain abstract and concrete methods.
Abstract method	Method without a body; concrete child class must implement it.
Interface	Defines a contract; class uses implements.
Multiple interfaces	A class can implement more than one interface.
Interface inheritance	An interface can extend one or more interfaces.
Default method	Interface method with body, inherited by implementing class.
Static method	Called using the interface name.
Private method	Helper method used inside the interface.
Functional interface	Exactly one abstract method.
Marker interface	No declared methods; marks a capability.
Package	Organises related classes and interfaces.
import	Uses types from another package by simple name.
static import	Uses static members without class qualification.
private	Accessible only within its declaring top-level class.
default	Accessible within the same package.
protected	Same package and qualifying subclasses outside it.
public	Accessible wherever the declaring type is accessible.
Classpath	Tells Java where to find classes and JAR files.

14. Important University Exam Questions

• Define an abstract class and abstract method with a Java program.

• What is an interface? Explain declaration and implementation.

• Explain multiple interface implementation with an example.

• What is interface inheritance? Can an interface extend multiple interfaces?

• Explain default, static and private methods in interfaces.

• Differentiate between functional and marker interfaces.

• Differentiate between an abstract class and an interface.

• What is a package? Explain built-in and user-defined packages.

• Explain import and static import with examples.

• Explain private, default, protected and public access modifiers.

• What is classpath? Explain how to set it.

• Write Java naming conventions with examples.

Exam writing tip: For a 5-mark answer, write the definition, 3–4 key points, a small syntax or program, and its output where suitable.